

como_instalar_servidor_git

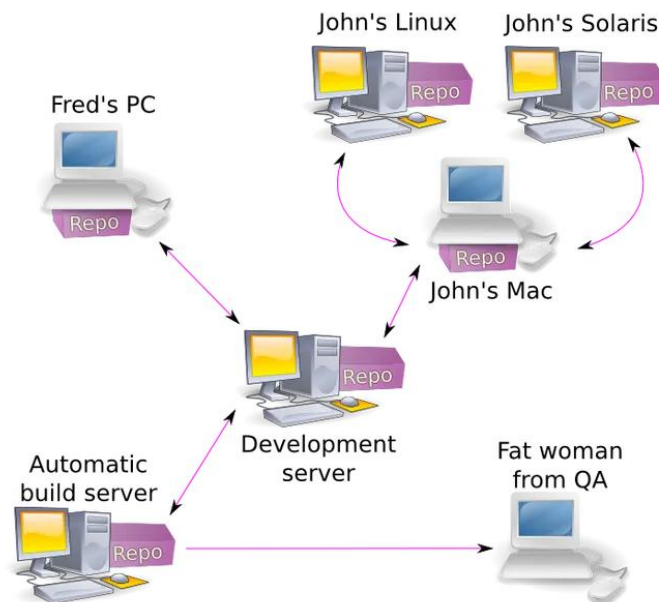
COMO INSTALAR E CONFIGURAR UM SERVIDOR GIT REMOTO ↗

- [COMO INSTALAR E CONFIGURAR UM SERVIDOR GIT REMOTO ↗](#)
 - [INTRODUÇÃO](#)
 - [INSTALANDO SERVIDOR GIT E SERVIDOR SSH NO DEBIAN OU DERIVADOS](#)
 - [CONFIGURAÇÕES DO REPOSITÓRIO NO SERVIDOR](#)
 - [COMO USAR O REPOSITÓRIO DO SERVIDOR NA MÁQUINA CLIENTE](#)
-

INTRODUÇÃO

1. Objetivo:

1. O *git* é um projeto criado por , [Linus Torvalds](#) cujo objetivo é criar um repositório de arquivos que possa ser mantido por várias pessoas. O *git* pode reproduzir várias versões de um projeto onde uma versão anterior possa ser gerada a qualquer momento caso seja necessário.
2. Basicamente, na maioria dos casos, o *Git* roda em um servidor que pode ser na *rede local* ou na *Web*. Nesse servidor mantemos um *repositório central*, o que vamos chamar de *Servidor Git*. Localmente, na sua estação de trabalho, por exemplo, é mantido um *repositório local*, ou seja, um *clone do repositório do Servidor Git*.
3. Nos arquivos dentro desse *repositório local* você realiza suas modificações e, ao concluí-las, as envia para o *servidor git*. Aquilo que modificou e seus comentários sobre suas mudanças são armazenados para consulta posterior.
4. Caso outras pessoas possuam um clone desse repositório, bastará rodar o comando *git pull* para obter as mudanças que você realizou e que já estarão armazenadas no *servidor git*.
 - **Nota:** Executar *git pull* antes de enviar modificações para o *servidor git*.
5. Na imagem abaixo podemos ver um exemplo de um time que pode ser de desenvolvimento, onde todos trabalham em um repositório local e enviam e recebem mudanças do repositório central (*servidor git*):



■

2. Pre-requisitos

1. Conhecimento básico de como se opera um terminal no sistema operacional Linux Debian ou derivados;
2. Conhecimento básico do projeto [Servidor OpenSSH](#) para entendimento de como funciona as *chaves ssh* necessárias para que o servidor acesse a máquina local cliente sem necessidade de pedir senha a cada atualização.
3. Uma máquina linux com Debian ou derivados para instalar o servidor git;
4. Uma máquina linux para testar o repositório criado no servidor git.

INSTALANDO SERVIDOR *GIT* E SERVIDOR *SSH* NO DEBIAN OU DERIVADOS

1. Comandos para instalar aplicativo *git*:

```
# Atualiza sistema lista do apt-get
sudo apt-get update

# Instala o git
sudo apt-get install git-all

# Trocar o nome padrão do branch para main
# Ao instalar o git o branch padrão é master, porém o github usa
main,
# por isso meus scripts de clientes estão main e o meu servidor
git deve acompanhar.
git config --global init.defaultBranch main
```

2. Comandos para instalar aplicativo *openssh-server*:

```
# Instalando o servidor ssh para que o usuário git client possa acessar
# o servidor sem pedir senha
apt-get install openssh-server
```

- **Notas:**

- Editar o arquivo `/etc/ssh/sshd_config` com direitos de root e informe a porta que o servidor `ssh` deve escutar.

```
# Obs: A porta 22 esta comentada, remova o comentário e informa porta que desejar.
sudo xed /etc/ssh/sshd_config
```

CONFIGURAÇÕES DO REPOSITÓRIO NO SERVIDOR

1. Criar um *usuário git* no servidor que será o proprietário dos repositórios a serem compartilhados com os clientes. Na prática, poderia ser qualquer usuário, mas para não ter que criar um usuário, no servidor, para cada cliente, é interessante usar o *usuário git*. Para criar esse usuário utilizamos o comando *useradd*:

```
sudo useradd --comment "Git user" --home-dir /home/git --groups users --shell $(which git-shell) git
```

- **Notas:**

1. Nesse exemplo o grupo do nosso *usuário git* é *users*. Esse grupo é o grupo padrão para a distribuição Debian. Caso seu servidor seja de outra distribuição coloque o respectivo grupo. Caso não saiba qual é esse grupo, verifique o do seu usuário e utilize-o.
 2. Nesse exemplo a opção *--shell* sendo atribuída ao utilitário *git-shell*. Fazemos isso para proibir o usuário *git* de se logar no nosso servidor. Ele somente consegue executar operações do programa *git*. Nada além disso. Isso garante que mesmo que um *hacker roube a senha* desse usuário ele jamais ganharia um *shell no servidor*.
2. Após criar o usuário, deve-se criar a pasta *home* dele, em seguida alterar o dono e grupo da pasta */home/git*.

```
# Cria a pasta inicial para o usuário git
sudo mkdir -p /home/git

# Conceda permissões de diretório ao usuário git
sudo chown -R git:users /home/git
```

3. Criar uma senha para o *usuário git*. Para isso iremos utilizar o *comando passwd*. Esse comando irá lhe pedir via prompt de comandos a nova senha:

```
# Definir senha do usuário git
sudo passwd git
```

4. Configurar nome e e-mail do *usuário git* globalmente:

```
# Definindo nome do usuário
git config --global user.name "paulospacheco"

# Definindo e-mail do usuário
git config --global user.email "paulospacheco@yahoo.com.br"
```

- **Nota**

- Se em algum momento desejar alterar as informações para um projeto específico, basta reescrever os comandos sem a opção *--global* dentro da pasta do projeto.

5. Configurando o protocolo **ssh**:

1. Quando a máquina cliente executa o comando *git clone hrl_do_repositório*, o *git* está utilizando o protocolo *ssh* para clonar o repositório remoto.
 - **Nota:** O protocolo HTTP também pode ser utilizado, mas não necessariamente essa comunicação estaria criptografada, autenticada e autorizada.
 - **Dica:** [Instalando e configurando servidor SSH \(Ubuntu\)](#);
2. Para configurar o *ssh* do servidor, é necessário editar o arquivo */etc/ssh/sshd_config* e adicionar a linha com a diretiva *AllowUsers* se não existir. No final da linha *AllowUsers*, adicione o nome do *usuário root* e *usuário git*. Siga os passos abaixo para fazer as alterações:
 1. Editar arquivo */etc/ssh/sshd_config*:

```
# Obs: Uso o editor xed , porém você pode usar que está disponível no momento.  
sudo xed /etc/ssh/sshd_config
```

2. Exemplo de como deve ficar a linha *AllowUsers* do arquivo */etc/ssh/sshd_config*:

```
# Authentication:  
# LoginGraceTime 2m  
# PermitRootLogin yes  
# StrictModes yes  
# MaxAuthTries 6  
# MaxSessions 10  
AllowUsers root git
```

3. Após gravar a alteração do arquivo */etc/ssh/sshd_config* com a adição do *usuário git* e *usuário root*, deve-se reiniciar o serviço no sistema operacional para que as modificações tenham efeito.

```
# System D  
sudo systemctl restart sshd.service  
  
# ou  
  
# Service  
sudo service sshd restart
```

- Dica: O processo *sshd* é um *daemon* em nosso servidor com a responsabilidade de receber as conexões vindas das máquinas clientes e garantir que estejam devidamente autorizadas ao logar no servidor.

3. Para acessar o repositório sem necessidade de digitar senha a todo momento é necessário seguir os seguintes passos:

1. Na *máquina servidora git*, seguir os passos abaixo, para que a máquina cliente não precise digitar senha para acessar o servidor:

1. Criar a pasta *ssh* para o *usuário git*:

```
sudo mkdir -p /home/git/.ssh/
```

2. Alterar o *nome do usuário* e do *grupo* para *git:users* na pasta invisível *./ssh*:

```
sudo chown -R git:users /home/git/.ssh
```

3. Altere as permissões da pasta */home/git/.ssh* para que somente o *usuário git* possa ler, gravar e executar na pasta:

```
sudo chmod 700 /home/git/.ssh
```

4. Cria o arquivo *authorized_keys* na pasta */home/git/.ssh* para cadastrar todas as chaves públicas das máquinas clientes com acesso aos repositórios:

```
sudo touch /home/git/.ssh/authorized_keys
```

5. Altera o *nome do usuário* e do *grupo* para *git:users* do arquivo *authorized_keys* da pasta */home/git/.ssh*:

```
sudo chown -R git:users  
/home/git/.ssh/authorized_keys
```

6. Altere as permissões do arquivo */home/git/.ssh/authorized_keys* para que somente o *usuário git* possa ler e gravar no arquivo:

```
sudo chmod 600 /home/git/.ssh/authorized_keys
```

7. Notas:

1. Além de criarmos a pasta *.ssh* e o arquivo *authorized_keys*, demos as devidas permissões a eles.
2. Para saber os parâmetros de autenticação do *git* execute o comando abaixo:

```
# Pega os parâmetro de acesso ao git
git config --list

# Resposta do comando acima:
> user.name=paulospacheco
> user.email=paulospacheco@yahoo.com.br
> core.autocrlf=input
> core.safecrlf=warn
```

2. Em cada *máquina cliente* que for acessar o servidor, executar os passos abaixo:
 1. Executar o programa *ssh-keygen* para criar um par de chaves (privada e pública) para poder enviar para o servidor a chave pública gerada.

```
# Move-se para a pasta invisível ~/.ssh
cd ~/.ssh

# O comando ssh-keygen criar o par de chave de
4096 bits cujo protocolo é rsa
ssh-keygen -t rsa -b 4096 -C
"your_email@yahoo.com.br"

# Em seguida, você vai ser solicitado a Inserir
arquivo no qual salvar a chave.
# Você pode especificar um local de arquivo ou
pressionar "Enter" para aceitar o local padrão do
arquivo.

> Enter a file in which to save the key
(/.ssh/id_rsa): [Press enter]

# O próximo prompt vai solicitar uma frase
secreta.
# A frase secreta vai adicionar uma camada
adicional de segurança ao SSH e vai ser necessária
sempre
# que a chave SSH for usada. Se alguém obtiver
acesso ao computador em que as chaves privadas estão
armazenadas,
# também vai poder obter acesso a qualquer sistema
que use essa chave. Adicionar uma frase secreta às
# chaves vai evitar esse cenário.

> Enter passphrase (empty for no passphrase):
[Type a passphrase]
> Enter same passphrase again: [Type passphrase
again]

# Pronto: Nesse ponto, vai ser gerada nova chave
SSH no caminho do arquivo especificado mais atrás.
```

■ **Notas:**

- Esse comando vai criar novo par de chaves SSH usando o e-mail como categoria.
- O comando `ssh-keygen` sugere que vai criar a chave `id_rsa` na pasta `~/.ssh`, porém salva o arquivo na pasta corrente. Passei um tempão para entender.

2. Executar o programa `ssh-add` para adicionar as chaves geradas pelo programa `ssh-keygen` em uma lista de chaves privadas. Além de manter chaves privadas, ele também controla solicitações SSH para que elas sejam transmitidas com segurança.

1. Antes de adicionar a nova chave SSH ao `ssh-agent`, primeiro verifique se o `ssh-agent` está sendo executado ao executar o

comando abaixo:

```
eval "$(ssh-agent -s)"  
> Agent pid 19895
```

■ Nota

- Se a resposta for *> Agent pid x* onde *x* é o número do processo, é porque está tudo ok.

2. *ssh-add* - Adiciona identidades de chave privada ao agente de autenticação *OpenSSH*

```
# Mova-se para a pasta ~/.ssh  
cd ~/.ssh  
  
# Adicionar a chave ./id_rsa ao ssh-agent  
ssh-add ./id_rsa  
> Enter passphrase for  
/home/paulospacheco/.ssh/id_rsa: # Obs: A  
senha é informada em ssh-keygen  
> Identity added:  
/home/paulospacheco/.ssh/id_rsa  
(paulospacheco@yahoo.com.br)
```

■ Notas

- A nova *chave SSH* agora está registrada e pronta para uso.
- A primeira vez que o *git* usa a chave, o sistema informa que a chave está bloqueada e precisa ser desbloqueada com a senha informada no programa *ssh-keygen*, ao criar a chave.
 - No *linux mint* é executado um diálogo no modo gráfico com dois campos, sendo 1 para a *senha* e o outro campo para o *flag* que informa que a senha deve ser permanente.
- Entre as duas chaves geradas, uma privada e outra pública, a chave com a extensão *.pub*, deve ser enviada para o servidor e ser adicionada ao final do arquivo */home/git/.ssh/authorized_keys* do servidor.

3. Após criar a chave na máquina *cliente git* a mesma deve ser registrada no *servidor git* executando os seguintes passos:

1. Você deve enviar a chave para o servidor pelo meio que estiver configurado na máquina cliente, podendo ser por *e-mail*, *WhatsApp*, *Telegram*, *pasta compartilhada* e etc...

- Suponha que a chave *id_rsa.pub* tenha sido salva na pasta *~/Downloads* do servidor, então executar os seguintes comandos:

```
# Se conecta como root
sudo -i

# Adiciona a chave no final do arquivo
/home/git/.ssh/authorized_keys
cat /home/git/.ssh/authorized_keys
/home/NomeSeuUsuário/Downloads/id_rsa.pub >>
/home/git/.ssh/authorized_keys

# Obs: Caso exista pelo menos uma chave o sistema
emitirá a mensagem abaixo
> cat: /home/git/.ssh/authorized_keys: os arquivos
de entrada e de saída são os mesmos
```

■ **Notas:**

- O comando acima ao adicionar a segunda chave, mostra um aviso informando que o arquivo de entrada e de saída são os mesmos, porém adiciona normalmente no final do arquivo a chave do arquivo *id_rsa.pub*.
- O processo acima também pode ser executado com qualquer editor de texto.

6. Antes de publicar o repositório para os clientes é necessário criá-lo usando a opção *--bare* do git:

1. O nome do repositório depende da necessidade de cada projeto, por exemplo: O *github*, usa uma conta de usuário e dentro de cada conta, usa o nome do repositório; porém este documento foi projetado pensando em apenas um usuário de nome *git*, por isso os comandos abaixo são para criar o repositório *test.git* dentro da pasta do *usuário git*:

```
# Crie a pasta test.git
sudo mkdir /home/git/test.git

# Mova-se para a pasta test.git
cd /home/git/test.git

# Inicializar repositório simples
sudo git init --bare

# Trocar o nome do usuário e do grupo dono da pasta
sudo chown -R git:users /home/git/test.git
```

■ **Notas:**

- Url para o repositório *test.git*:
 - nome: origin
 - url: <git@192.168.15.3:/home/git/test.git>

7. Como saber o link que o *cliente git* deve usar para clonar o repositório do *servidor git*?

1. Cada projeto tem sua estrutura de árvore dentro do servidor git, por exemplo, o *github* usa a seguinte sintaxe: <git@github.com:Usuário/NomeDoRepositório.git>

```
# Exemplo de repositório do github
git@github.com:paulospacheco/blog.pssp.app.br.git
```

2. Os exemplos deste documento usa a seguinte sintaxe:
git@ip_server:/home/git/repositório.git:

```
# Exemplo repositório na rede local: ip=192.168.15.3
git@192.168.15.3:/home/git/test.git
```

8. Depois de executar todas as tarefas acima a URL

<git@192.168.15.3:/home/git/test.git> pode ser publicada para os clientes.

COMO USAR O REPOSITÓRIO DO SERVIDOR NA MÁQUINA CLIENTE

1. O que o *cliente git* deve fazer para clonar o repositório

<git@192.168.15.3:/home/git/test.git>

1. [instalar o git na máquina cliente](#);
2. Criar uma *chave ssh* privada para que o servidor git permita enviar arquivos sem necessidade de senhas.

- Esses passos foi descrito no tópico [5.3.2](#) deste documento.

3. A *chave ssh* tem duas partes, sendo um arquivo com a *chave privada* e um arquivo com a *chave pública*. É necessário registrar no servidor a chave pública da máquina cliente para que o servidor saiba que as solicitações do cliente é segura.

- Esses passos foi descrito no tópico [5.3.3](#) deste documento

4. Clonar o repositório [git@192.168.15.3:/home/git/test.git](#) na máquina cliente:

```
# Clonar o repositório test
git clone git@192.168.15.3:/home/git/test.git
```

- **Notas:**

- O comando acima irá criar uma pasta na máquina cliente com todos os dados do repositório no servidor;

2. O que o *cliente git* deve fazer antes de editar os arquivos do repositório local?

```
# Atualiza o repositório local com os dados do repositório remoto
git pull
```

- O comando acima não deve ser automático, executando antes do comando *git push -u origin main*, porque a versão *main remota* pode estar desatualizada.
- O comando *git pull* deve ser executado antes das alterações do repositório local.

3. O que o *cliente git* deve fazer após alterar os arquivos do repositório local para atualizar repositório remoto?

```
# Associa o repositório remoto ao repositório local.
git remote add origin git@192.168.15.3:/home/git/test.git

# Renomeie o branch atual para main
# O comando branch -M não precisa ser feito a todo momento,
porque o git sempre envia para
# o ultimo ramo selecionando.
git branch -M main

# Este comando pode ser executado várias vezes antes de um
commit.
git add .

# Use o <msg> fornecido como a mensagem de confirmação.
git commit -a -m "Texto descrevendo as alterações realizadas"

# Envia as alterações locais para o repositório remoto.
git push -u origin main

# imprime o status atual do repositório
git status
```

